Project 1
6 DOF Robotic Arm Using Joystick Control

Overview
This project involves designing and building a 6 Degrees of Freedom (DOF) Robotic Arm controlled using two joystick modules. The arm is powered by a combination of RC digital servo motors and SG90 servo motors. The base, shoulder, and elbow joints use RC digital servos with an external power source, while the wrist, gripper, and gripper rotation use SG90 servos powered by the Arduino. The system enables precise movement and flexibility for robotic applications.

Components Used
- Arduino Uno – Microcontroller for processing inputs and controlling servos.
- RC Digital Servo Motors (35kg) – Used for the base, shoulder, and elbow joints, powered by an external voltage source.
- SG90 Servo Motors – Used for wrist, gripper, and gripper rotation.
- Joystick Modules (x2) – Used to control different movements of the robotic arm.
- HC-10 Bluetooth Module – Allows wireless control (if required in future enhancements).
- External Power Supply – Powers the high-torque servo motors.

Working Principle
1. The joysticks send analog signals to the Arduino Uno, which processes them to control the servos.
2. The base, shoulder, and elbow servos (RC digital) handle the major movements and require a stable external power source.
3. The wrist, gripper, and gripper rotation servos (SG90) receive power from the Arduino itself.
4. The gripper toggles between open and closed states using a button on the joystick.
5. A reset button brings the arm back to its default position.

Circuit Connections
- Base Servo (RC Digital) – PWM Pin 3 (External Power Supply)
- Shoulder Servo (RC Digital) – PWM Pin 5 (External Power Supply)
- Elbow Servo (RC Digital) – PWM Pin 6 (External Power Supply)
- Wrist Servo (SG90) – PWM Pin 9 (Arduino Power)
- Gripper Servo (SG90) – PWM Pin 10 (Arduino Power)
- Gripper Rotation Servo (SG90) – PWM Pin 11 (Arduino Power)
- Joystick 1:
  - VRX1 → A0 (Base Movement)
  - VRY1 → A1 (Shoulder Movement)
  - Button1 → Pin 7 (Gripper Toggle)

- Joystick 2:
  - VRX2 → A2 (Gripper Rotation)
  - VRY2 → A3 (Wrist Movement)
  - Button2 → Pin 8 (Reset Arm Position)

Arduino Code

```
#include <Servo.h>

Servo baseServo, shoulderServo, elbowServo, wristServo, gripperServo,
gripperRotationServo;

#define VRX1 A0
#define VRY1 A1
#define BUTTON1 7
#define VRX2 A2
#define VRY2 A3
#define BUTTON2 8

int baseAngle = 90, shoulderAngle = 90, elbowAngle = 90, wristAngle = 90,
   gripperAngle = 0, gripperRotationAngle = 90;

bool lastButtonState1 = HIGH;
bool lastButtonState2 = HIGH;
bool gripperOpen = false;


#define DEADZONE 50
#define STEP_SIZE 2

void setup() {
   Serial.begin(115200);
   baseServo.attach(3);
   shoulderServo.attach(5);
   elbowServo.attach(6);
   wristServo.attach(9);
   gripperServo.attach(10);
   gripperRotationServo.attach(11);
   pinMode(BUTTON1, INPUT_PULLUP);
   pinMode(BUTTON2, INPUT_PULLUP);
}

void loop() {
   int xValue1 = analogRead(VRX1);
```

```
  int yValue1 = analogRead(VRY1);
  int xValue2 = analogRead(VRX2);
  int yValue2 = analogRead(VRY2);

  bool buttonState1 = digitalRead(BUTTON1) == LOW;
  bool buttonState2 = digitalRead(BUTTON2) == LOW;

  if (buttonState1 && !lastButtonState1) {
    gripperOpen = !gripperOpen;  // Toggle state
    gripperAngle = gripperOpen ? 120 : 0;
    gripperServo.write(gripperAngle);
    Serial.print("Gripper: "); Serial.println(gripperAngle);
    delay(200);
  }
  lastButtonState1 = buttonState1;

  if (buttonState2 && !lastButtonState2) {
    resetPosition();
    Serial.println("Arm position reset!");
    delay(300);
  }
  lastButtonState2 = buttonState2;

  if (abs(xValue1 - 512) > DEADZONE) {
    int targetAngle = map(xValue1, 0, 1023, 0, 180);
    baseAngle = moveGradually(baseAngle, targetAngle);
  }
  if (abs(yValue1 - 512) > DEADZONE) {
    int targetAngle = map(yValue1, 0, 1023, 0, 180);
    shoulderAngle = moveGradually(shoulderAngle, targetAngle);
  }
  if (abs(xValue2 - 512) > DEADZONE) {
    int targetAngle = map(xValue2, 0, 1023, 0, 180);
    gripperRotationAngle = moveGradually(gripperRotationAngle, targetAngle);
  }
  if (abs(yValue2 - 512) > DEADZONE) {
    int targetAngle = map(yValue2, 0, 1023, 0, 180);
    wristAngle = moveGradually(wristAngle, targetAngle);
  }
  moveServos();
  delay(20);
}
```

```
void resetPosition() {
  baseAngle = shoulderAngle = elbowAngle = wristAngle = gripperRotationAngle = 90;
  gripperAngle = 0;
  gripperOpen = false;
  moveServos();
}

void moveServos() {
  baseServo.write(baseAngle);
  shoulderServo.write(shoulderAngle);
  elbowServo.write(elbowAngle);
  wristServo.write(wristAngle);
  gripperRotationServo.write(gripperRotationAngle);
  gripperServo.write(gripperAngle);
}
```

Project 2

Page 1

4-BIT PALINDROME DETECTOR

USING FSM AND VERILOG HDL

Simple Project Report

Prepared by: Sai Divesh Reddy Karri

Roll No.: 2K24ECUN03025

This report explains the project in a simple way so that the working of the FSM and Verilog code can be

understood easily.

Page 2

1. What is a Palindrome Detector?

A palindrome is a sequence that remains the same when read from both directions. In this project, we

check whether a 4-bit binary number is a palindrome.

Examples:

0000 ® Palindrome

0110 ® Palindrome

1001 ® Palindrome

1111 ® Palindrome

0101 ® Not a palindrome

0011 ® Not a palindrome

2. Objective

The main objective is to design a circuit that receives four binary bits one by one and checks whether the

complete 4-bit sequence is a palindrome. The circuit is designed using a Finite State Machine (FSM) and

Verilog HDL.

3. Basic Idea

For a 4-bit sequence B3 B2 B1 B0 to be a palindrome, the first and last bits must be equal, and the two

middle bits must also be equal.

Condition:

B3 = B0

B2 = B1

So, if both conditions are true, the output is 1 (Palindrome). Otherwise, the output is 0 (Not Palindrome).

4. How the FSM Works

The FSM receives one bit during each clock cycle. It moves through five states:

State What happens

S0 Initial state / start

S1 First bit is stored

S2 Second bit is stored

S3 Third bit is stored

S4 Fourth bit is stored and palindrome is checked

The basic flow is:

S0 ® S1 ® S2 ® S3 ® S4 ® S0

After the fourth bit is received, the stored bits are compared. The FSM then returns to S0 to process the

next sequence.

Page 3

5. Verilog Code

A simple FSM-based Verilog implementation is shown below:

```
module palindrome_detector (

input wire clk,

input wire rst,

input wire data_in,

output reg palindrome

);

reg [2:0] state;

reg [3:0] data;

parameter S0 = 3'b000,

S1 = 3'b001,

S2 = 3'b010,

S3 = 3'b011,

S4 = 3'b100;

always @(posedge clk or posedge rst) begin

if (rst) begin

state <= S0;

data <= 4'b0000;

palindrome <= 1'b0;

end

else begin

case (state)
```

```
S0: begin
data[3] <= data_in;
state <= S1;
end
S1: begin
data[2] <= data_in;
state <= S2;
end
S2: begin
data[1] <= data_in;
state <= S3;
end
S3: begin
data[0] <= data_in;
state <= S4;
end
S4: begin
if ((data[3] == data[0]) &&
(data[2] == data[1]))
palindrome <= 1'b1;
else
palindrome <= 1'b0;
state <= S0;
end
default: begin
state <= S0;
palindrome <= 1'b0;
end
```

endcase

end

end

endmodule

6. Example

Page 4

Suppose the input bits are received as:

0 ® 1 ® 1 ® 0

The complete sequence is 0110. Its reverse is also 0110, so it is a palindrome and the output becomes 1.

For 0101, the reverse is 1010. Therefore it is not a palindrome and the output is 0.

7. Test Cases

Input Output

0000 Palindrome (1)

0110 Palindrome (1)

1001 Palindrome (1)

1111 Palindrome (1)

0101 Not Palindrome (0)

0011 Not Palindrome (0)

8. Result

The FSM successfully receives four serial input bits and checks whether the resulting sequence is a palindrome. The expected results are obtained for both palindrome and non-palindrome test cases.

9. Conclusion

This project helped demonstrate how a Finite State Machine can be used in a practical digital circuit. Using Verilog HDL, the 4-bit input is collected over multiple clock cycles and then checked for the palindrome condition. The project also gives a basic understanding of RTL design, state transitions, sequential logic, and simulation.